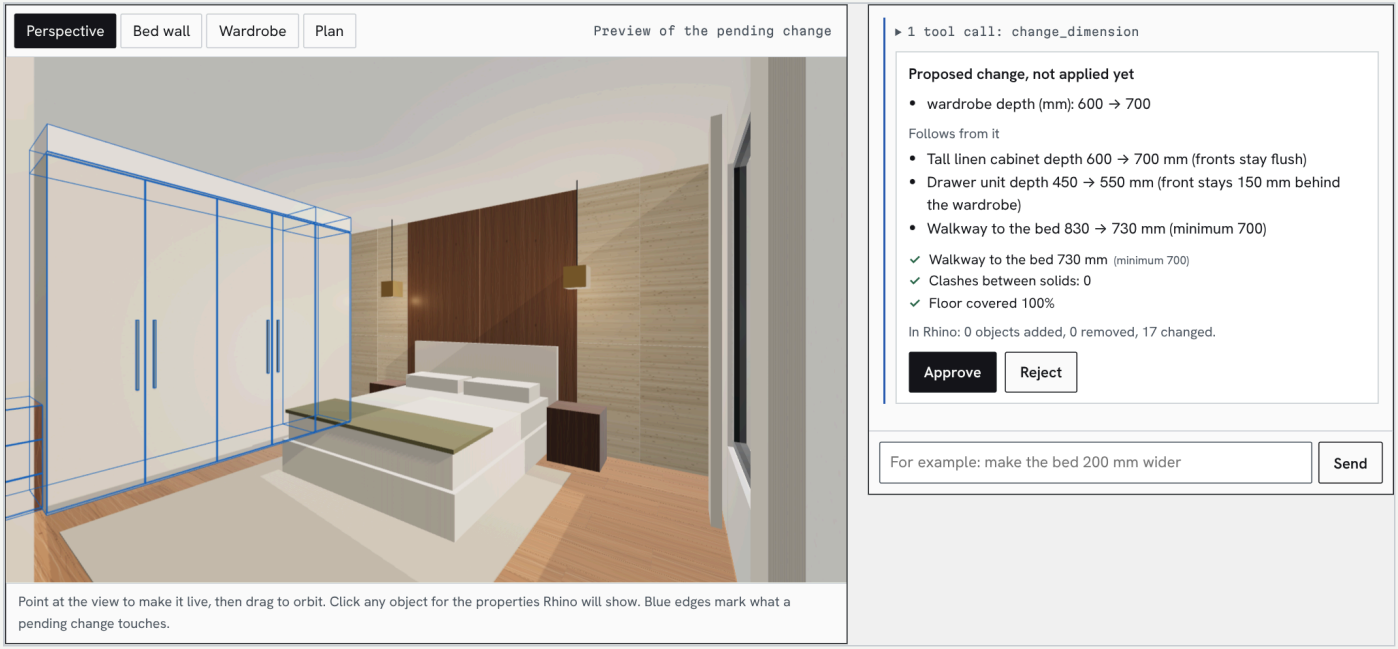
The five instructions from your brief, applied to an editable Rhino model.

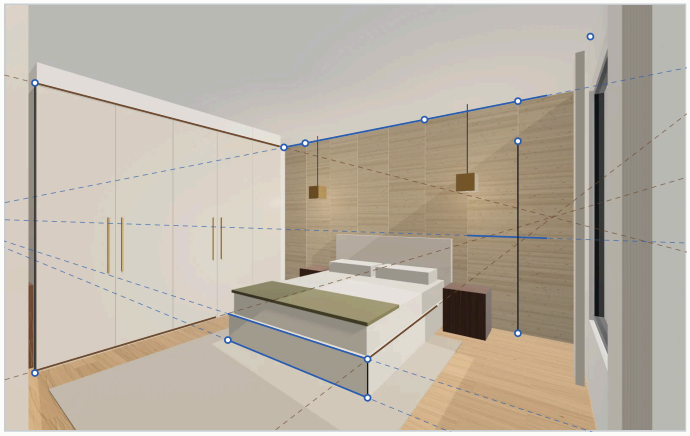
I built the proof of concept before writing this. A sample room is reconstructed from a rendering, the agent turns each instruction into typed tool calls, every change is checked and shown before it is applied, and each version is a .3dm file that opens in Rhino. Everything below was measured; what was not is listed on page 2.



Instruction 3 in the demo: the wardrobe change is previewed with blue edges, its knock-on effects and checks listed, and nothing changes until Approve.

Phase 1. From a rendering to an editable model

A–101 Measured on a test rendering



29 clicked points, all checked to be visible. Edges along the two room axes give the vanishing points, the lens and the camera; the ceiling height from the drawings gives the scale.

At 1 px click error, 95% within	True	±mm
Room width (wall to wall)	4,800	83
Wardrobe front from wall	620	136
Wardrobe run to feature wall	3,680	32
Wardrobe height	2,700	47
Bed width	1,900	36
Bed + headboard length	2,230	32
Bed centre from west wall	2,400	103
Bed height	320	9
Travertine slab width	600	15
Travertine slab height	1,200	9

- 8 of 10 dimensions within ±2.9%. Positions taken from the west wall, seen edge-on, are weak: wardrobe depth comes from the joinery drawing.
- Without drawings, a 3,000 mm ceiling guess makes the room 167 mm too wide.
- Lens shift: 320 mm error if the image centre is assumed, 0.1 mm when taken from the horizon.

Phase 2. The agent and the Rhino connection

A–201 Built and tested outside Rhino

Designer	OpenAI model	App	Rhino 8 over MCP	Versions
instruction, approve or reject	typed tool calls from 9 strict schemas, never code	validates, applies to the spec, runs checks, shows the diff	7 reviewed Python functions, one undo record per change	.3dm with the spec inside, renders, rollback

> **Risk in the plan:** in McNeel's Rhino AI, unchecking the destructive tools (scripts, save, run commands) only hides them from agents in Rhino's own panel. An external client connected over MCP still sees every tool. So GPT or Codex never connects to Rhino here; the app is the only MCP client and sends plain data to fixed functions.

Checks before Rhino is touched: wardrobe depth range, plank width range, feature wall zones adding up, a 700 mm walkway to the bed, clashes, floor coverage. Example: *Increase the wardrobe depth by another 300 mm.* was blocked (walkway 430 mm) and the agent offered +30 mm. instead.

Held-out phrasings	14/20 exact with the offline parser, run once; 4 asked a question, 1 partial, 1 disputed label
Written phrasings	50/50
OpenAI path	same tool schema; eval script ready, needs your key

Before every change the app checks for manual edits in Rhino: a 50 mm move, a deleted object and an extra object were all detected on a real .3dm.

Phase 3. Renders, options and versions

A–301 Built; images are previews



Version 1, reconstructed



Version 2: walnut 240 mm planks, oak panels between travertine, wardrobe 700 mm



Option A: Fluted oak centre



Option B: Lime plaster with walnut shelf, silver travertine sides



Option C: Full-width smoked oak panels

5 .3dm files written, opened again with McNeel's rhino3dm in Python: millimetres, every object a closed solid on a named layer with element, part and material user text, and 0 mm difference when rebuilt from the embedded spec by the generator that runs in Rhino.

Phase 4. Tests, handover

A–401

One rendering reconstructed	method and error measured; yours in the PoC
Agent connected	offline parser here; OpenAI and your Rhino 8 in the PoC
Three conversational changes	instructions 1 to 3 approved, one blocked with a reason
Updated visualisations	previews here; Rhino Render from saved views in the PoC
Saved editable model	.3dm per version, read back and checked

20 unit, 6 Python and 7 browser tests pass, including a version saved in the browser and opened with rhino3dm. Handover: source and tests, Rhino setup guide, material library and rules as files, recorded walkthrough and a training session. You own all of it.

Measured

- Dimension recovery on a synthetic rendering with known truth (2,000 runs per error level)
- .3dm structure and rebuild; manual-edit detection
- Instruction parsing; checks, approvals, versions and rollback

Not measured

- Running inside Rhino 8 over MCP (no licence here; the Rhino functions are written, not run)
- Real client renderings, OpenAI accuracy and cost, Rhino Render timing
- Curved or freeform elements

Sources

- rhino3d.com/for/ai: tool groups and MCP clients
- docs.mcneel.com: Rhino 9 AI command and AI Settings
- discourse.mcneel.com: RhinoAI beta feature